

Техническая документация мобильного приложения FarmerApp

Система прямой продажи фермерских товаров

Документ	Техническое описание системы
Автор проекта	Малородов Михаил
Год	2026

Оглавление

1	Назначение документа	4
1.1	Область системы	4
1.2	Термины	4
2	Участники и варианты использования	5
2.1	Матрица доступа	5
3	Требования к системе	7
3.1	Пользовательские требования	7
3.1.1	Покупатель	7
3.1.2	Фермер	7
3.2	Функциональные требования	7
3.3	Нефункциональные требования	8
4	Бизнес процессы	10
4.1	Существующий процесс	10
4.2	Целевой процесс	10
5	Архитектура системы	12
5.1	Компоненты	12
5.2	Поток запроса	13
6	Клиентское приложение	14
6.1	Навигация и состояние	14
6.2	Локальные данные	14
7	Модель данных	15
7.1	Сущность users	15
7.2	Сущность products	16
7.3	Сущность points_of_sale	16
7.4	Сущность product_images	17
7.5	Сущность orders	17
7.6	Сущность order_items	17
7.7	Сущность messages	18
7.8	Сущность reviews	18
7.9	Ограничения целостности	18
8	Программные интерфейсы	19
8.1	Правила контракта	20
9	Безопасность и конфигурация	21

9.1	Безопасный пример application properties	21
10	Сборка и запуск	22
10.1	Сервер	22
10.2	Android клиент	22
11	Тестирование	23
11.1	Функциональные сценарии покупателя	23
11.2	Функциональные сценарии фермера	23
11.3	Модульные тесты	23
11.4	Результаты нагрузочного теста	24
12	Сопровождение и публикация	25
12.1	Правила обновления документации	25
12.2	Пункты для сверки с исходным кодом	25
13	Приложение диаграммы	26

1 Назначение документа

Документ описывает устройство FarmerApp, требования к системе, бизнес-процессы, архитектуру, модель данных, программные интерфейсы, правила безопасной конфигурации, развертывание и проверку качества. Он предназначен для разработчиков, руководителя дипломного проекта и специалистов, которые будут сопровождать приложение после передачи исходного кода.

CaseFarmer реализует прямое взаимодействие между покупателем и фермером. Покупатель находит локальные товары, выбирает точку самовывоза, формирует бронирование и общается с продавцом. Фермер управляет ассортиментом, точками продаж и заказами. Мобильный клиент работает с сервером Spring Boot, базой PostgreSQL, картами Яндекса и Firebase Cloud Messaging.

1.1 Область системы

- Android-приложение с отдельными сценариями покупателя и фермера.
- Каталог фермерских товаров с поиском, фильтрацией и постраничной загрузкой.
- Локальная корзина и создание бронирований с разбиением по фермерам.
- Управление товарами, точками продаж и статусами заказов.
- Отображение активных точек продаж на карте и работа с геолокацией.
- Чат по заказу через WebSocket и push-уведомления через FCM.

1.2 Термины

Термин	Значение
Покупатель	Пользователь, который ищет товары, создает бронирование и получает заказ.
Фермер	Пользователь, который публикует товары, управляет точками продаж и обрабатывает заказы.
Точка продаж	Физическое место выдачи или продажи, привязанное к координатам и ассортименту.
Бронирование	Заказ на товары конкретного фермера без встроенной оплаты и доставки.
Активная точка	Точка продаж, товары которой доступны покупателям в каталоге и на карте.
D2C	Модель прямого взаимодействия производителя с покупателем без торгового посредника.

2 Участники и варианты использования

В системе предусмотрены две прикладные роли. Общие функции регистрации и входа доступны обеим ролям; операции с каталогом и бронированием разделены по назначению. Чат создается в контексте заказа и связывает только его покупателя и фермера.

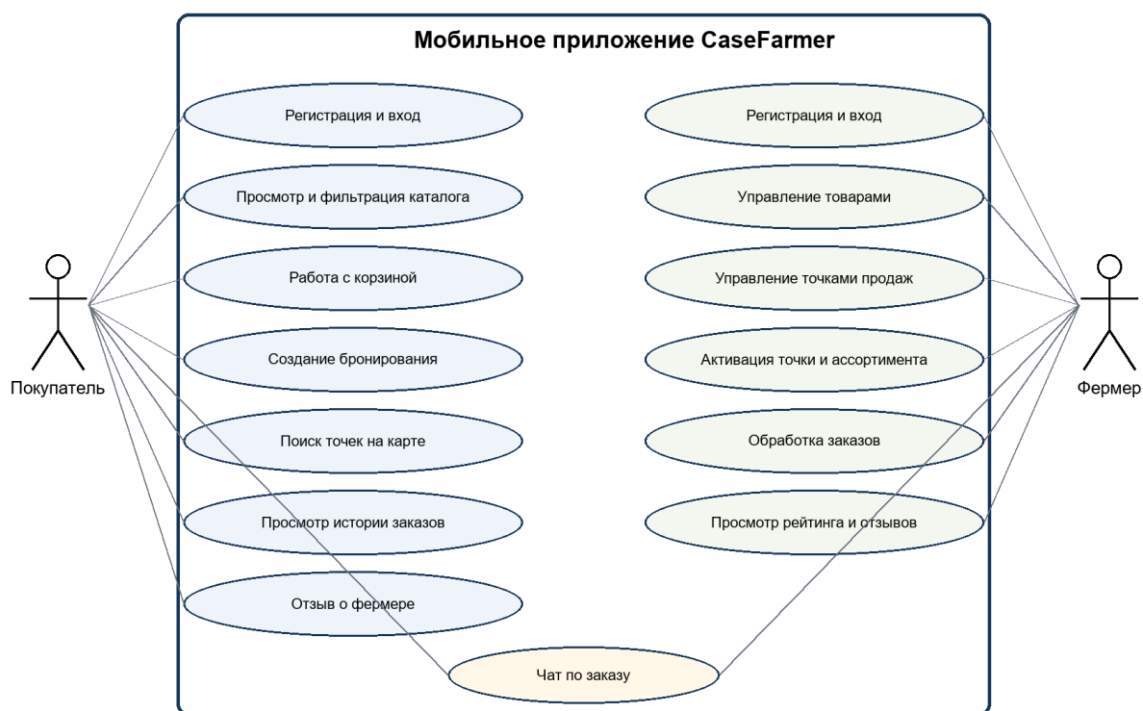


Рисунок 1 - Варианты использования FarmerApp

2.1 Матрица доступа

Функция	Покупатель	Фермер
Регистрация и вход	Да	Да
Просмотр и фильтрация каталога	Да	Нет
Корзина и создание бронирования	Да	Нет
Просмотр карты активных точек	Да	Нет
Просмотр истории своих заказов	Да	Да
Управление собственными товарами	Нет	Да
Управление собственными точками продаж	Нет	Да
Подтверждение или отклонение заказа	Нет	Да
Подтверждение получения или отмена до подтверждения	Да	Нет

Функция	Покупатель	Фермер
Чат по активному заказу	Да	Да
Отзыв о фермере	Да	Просмотр

3 Требования к системе

3.1 Пользовательские требования

3.1.1 Покупатель

- Быстро находить товары по названию, категории, цене и расположению точки продаж.
- Просматривать описание, цену, фотографии и сведения о продавце до обращения к фермеру.
- Сохранять товары в корзину и создавать бронирование.
- Видеть активные точки продаж на карте и оценивать расстояние до них.
- Просматривать историю заказов и общаться с фермером внутри заказа.
- Оставлять отзыв после выполнения заказа и учитывать рейтинг продавца при выборе.

3.1.2 Фермер

- Создавать карточку товара с описанием, ценой, количеством и фотографиями.
- Редактировать и удалять собственные товары.
- Создавать точки продаж, назначать им координаты и управлять активностью.
- Выбирать ассортимент активной точки продаж.
- Получать сведения о новых заказах и менять их статус.
- Общаться с покупателем внутри заказа и просматривать рейтинг и отзывы.

3.2 Функциональные требования

Код	Требование	Приоритет
FR-AUTH-01	Регистрация покупателя по имени, email и паролю с подтверждением email	Высокий
FR-AUTH-02	Регистрация фермера с названием фермы, именем, email и паролем	Высокий
FR-AUTH-03	Вход по email и паролю с выдачей JWT	Высокий
FR-AUTH-04	Восстановление пароля через email	Средний
FR-AUTH-05	Выход с удалением JWT на клиенте	Высокий
FR-AUTH-06	Просмотр и редактирование профиля	Низкий
FR-AUTH-07	Зашифрованное хранение JWT на устройстве	Высокий
FR-PROD-01	Добавление товара с названием, категорией, ценой, количеством, описанием и фото	Высокий
FR-PROD-02	Загрузка не более трех фотографий товара	Высокий
FR-PROD-03	Редактирование собственного товара	Средний
FR-PROD-04	Удаление собственного товара	Средний
FR-PROD-05	Просмотр списка собственных товаров	Высокий
FR-CAT-01	Просмотр активных товаров с пагинацией	Высокий
FR-CAT-02	Поиск по названию	Высокий

Код	Требование	Приоритет
FR-CAT-03	Фильтрация по цене, категории и расстоянию	Высокий
FR-CAT-04	Просмотр подробной информации о товаре	Высокий
FR-CAT-05	Добавление товара в локальную корзину	Высокий
FR-ORD-01	Создание бронирования покупателем	Высокий
FR-ORD-02	Просмотр списка заказов и их статусов	Высокий
FR-ORD-03	Просмотр состава, суммы, причины отмены и истории переписки	Высокий
FR-ORD-04	Подтверждение или отклонение заказа фермером	Высокий
FR-ORD-05	Подтверждение получения покупателем	Высокий
FR-ORD-06	Статусы ожидания, подтверждения, отмены и завершения	Высокий
FR-ORD-07	Разбиение корзины на отдельные заказы по фермерам	Высокий
FR-ORD-08	Запрет новых сообщений после закрытия заказа	Высокий
FR-MAP-01	Отображение текущего местоположения пользователя	Высокий
FR-MAP-02	Поиск места по адресу или координатам	Высокий
FR-MAP-03	Отображение расстояния до точек продаж	Средний
FR-CHAT-01	Создание чата при создании заказа	Высокий
FR-CHAT-02	Обмен текстовыми сообщениями и фотографиями в реальном времени	Высокий
FR-CHAT-03	Хранение и загрузка истории сообщений	Высокий
FR-CHAT-04	Статус прочтения сообщений	Низкий
FR-CHAT-05	Push-уведомления о новых сообщениях и изменении заказа	Средний

3.3 Нефункциональные требования

Категория	Показатель	Цель	Проверка
Производительность	Отклик API	Менее 1 секунды	Замер p50, p95 и максимума под согласованной нагрузкой
Производительность	Загрузка каталога	Менее 4 секунд	Замер на поддерживаемом устройстве и сети
Производительность	История заказов	Менее 10 секунд	Замер на заполненном наборе данных
Производительность	Профиль	Менее 1,5 секунды	Замер полного отображения экрана
Масштабирование	Одновременные пользователи	Не менее 1000	Нагрузочный сценарий с основными эндпоинтами
Надежность	Доступность	Простой менее 1 процента в месяц	Мониторинг доступности API

Категория	Показатель	Цель	Проверка
Надежность	Ошибки сети и сервера	Понятные сообщения и повтор операции	Тесты отключения сети и ответов 4xx и 5xx
Безопасность	Пароли	Только стойкий хеш	Проверка схемы хранения и настроек кодировщика
Безопасность	Защищенные ресурсы	Валидный JWT и ролевая проверка	Негативные тесты без токена и с чужой ролью
Совместимость	Android	Минимальная версия должна совпадать с minSdk	Проверка Gradle и матрицы устройств
Совместимость	Экран	От 320dp до 1440dp, портретная ориентация	Снимки на нескольких конфигурациях
Локализация	Язык интерфейса	Русский	Проверка отсутствующих строковых ресурсов

4 Бизнес процессы

4.1 Существующий процесс

Модель AS IS отражает разрозненный процесс поиска и продажи. Фермер готовит товар, публикует объявления на нескольких площадках и отвечает на повторяющиеся запросы. Покупатель ищет сведения на площадках, в социальных сетях, на рынках и через знакомых, затем вручную проверяет наличие и репутацию продавца. Согласование цены, объема, времени и места встречи проходит в сторонних каналах. Сделка может не состояться, а ее история не сохраняется в единой системе.

- Дублирование объявлений и ручное обновление наличия.
- Фрагментированный поиск и отсутствие единого каталога.
- Ручная проверка продавца и товара.
- Нет общей истории договоренностей и статусов бронирования.
- Точка продажи и актуальный ассортимент не связаны в одном интерфейсе.

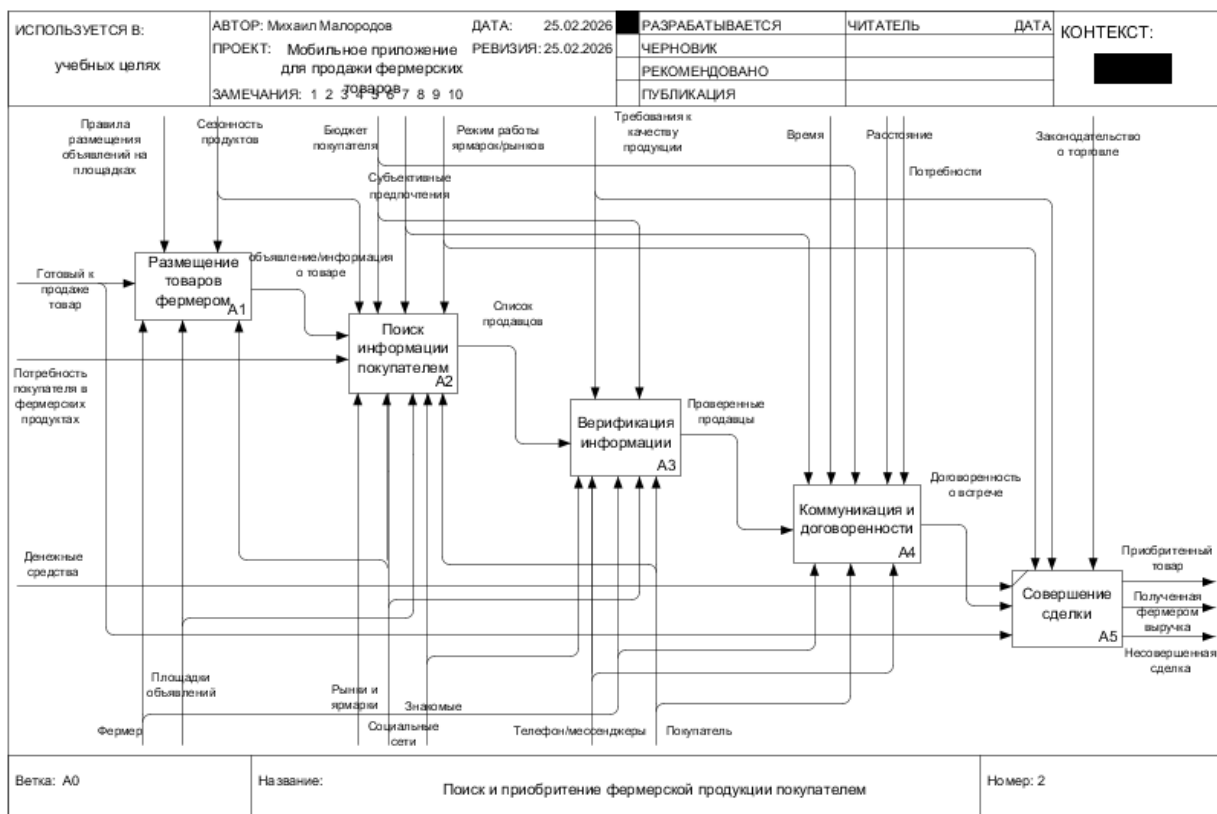


Рисунок 2 - Декомпозиция процесса AS IS

4.2 Целевой процесс

В модели TO BE публикация, поиск, бронирование и коммуникация выполняются внутри CaseFarmer. Фермер ведет профиль, каталог и точки продаж; покупатель получает единый список активных товаров, создает бронирование и использует чат заказа. Сервер контролирует доступ, сохраняет состояние в PostgreSQL и инициирует уведомления.

1. Пользователь регистрируется, подтверждает email и получает JWT после входа.

2. Фермер создает товары и точку продаж, выбирает доступный ассортимент и активирует точку.
3. Покупатель ищет и фильтрует товары либо выбирает активную точку на карте.
4. Корзина разделяется по фермерам, после чего сервер создает отдельные заказы.
5. Фермер подтверждает или отклоняет заказ, а покупатель получает уведомление.
6. Участники уточняют детали в чате, связанном с заказом.
7. После передачи товара заказ завершается и может стать основанием для отзыва.

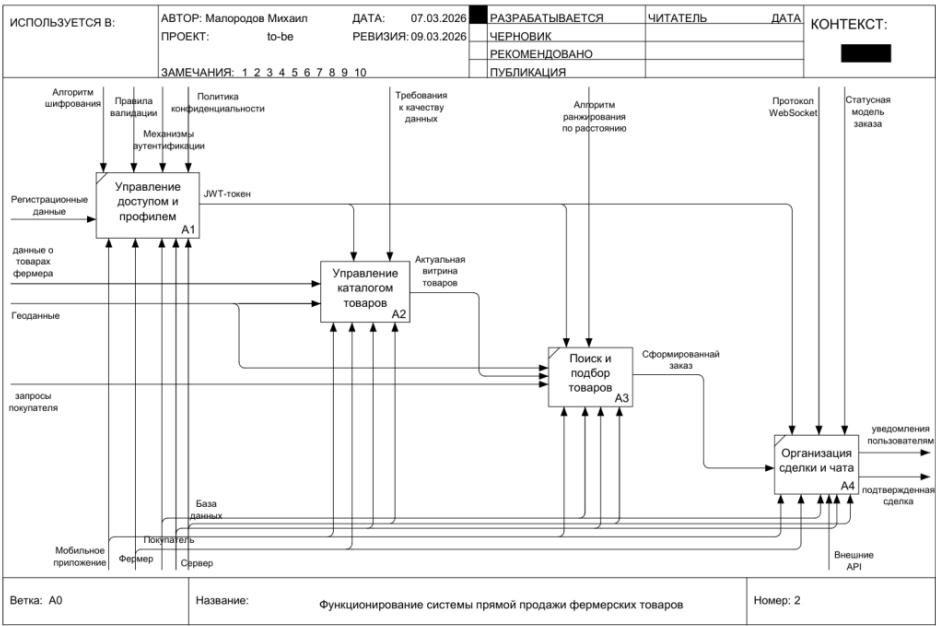


Рисунок 3 - Декомпозиция процесса TO BE

5 Архитектура системы

Система построена по клиент-серверной схеме. Android-клиент отвечает за интерфейс, локальное состояние и обращение к внешним SDK. Сервер Spring Boot реализует бизнес-логику, авторизацию, REST API и WebSocket. PostgreSQL хранит прикладные данные. Yandex MapKit и Firebase Cloud Messaging подключаются как внешние сервисы.

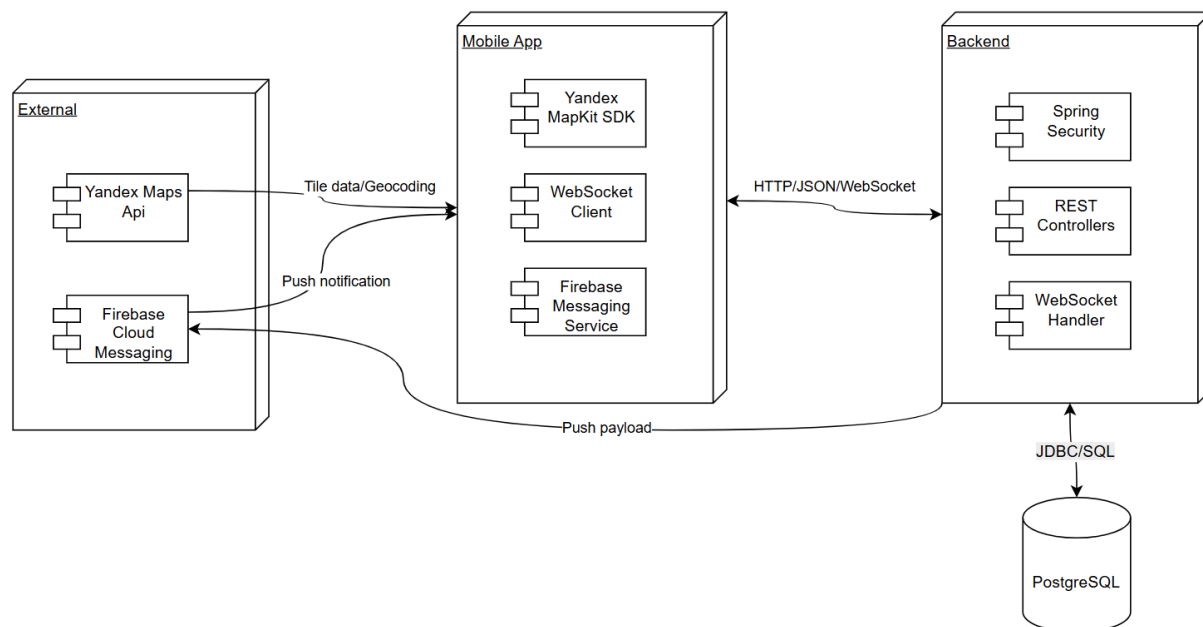


Рисунок 4 - Архитектура CaseFarmer

5.1 Компоненты

Компонент	Технологии	Ответственность
Android-клиент	Kotlin, MVVM, Fragment, Flow, LiveData	Интерфейс, состояние экрана, навигация, валидация и вызов серверных интерфейсов
Сетевой слой клиента	Retrofit, WebSocket Client	HTTP и JSON-запросы, JWT-заголовок, события чата
Локальное хранение	Room, DataStore, security-crypto	Корзина, настройки и зашифрованный токен
Сервер	Java 17, Spring Boot, Spring Security	Бизнес-правила, ролевая проверка, REST API и управление заказами
Доступ к данным	Spring Data JPA, Hibernate, PostgreSQL	Транзакции и долговременное хранение
Карты	Yandex MapKit, Location API	Карта, геокодирование, координаты и местоположение
Уведомления	Firebase Cloud Messaging	Push-уведомления о сообщениях и статусах заказов

Компонент	Технологии	Ответственность
Изображения	Multipart, Glide	Передача, загрузка и кэширование фотографий товаров

5.2 Поток запроса

1. Экран передает пользовательское действие во ViewModel.
2. ViewModel обращается к репозиторию и публикует состояние через Flow или LiveData.
3. Репозиторий выбирает локальный источник Room либо сетевой источник Retrofit.
4. Retrofit отправляет JSON или multipart-запрос и прикладывает JWT к защищенному ресурсу.
5. Spring Security проверяет токен и роль до передачи запроса контроллеру.
6. Сервис выполняет бизнес-правило, а репозиторий JPA изменяет данные в транзакции.
7. Клиент получает DTO, обновляет состояние и перерисовывает только изменившиеся элементы списка.

6 Клиентское приложение

6.1 Навигация и состояние

Приложение использует одну Activity и отдельные графы навигации для авторизации, покупателя и фермера. BottomNavigationView предоставляет быстрый доступ к основным разделам роли. ViewModel сохраняет состояние экрана, а Flow и repeatOnLifecycle ограничивают сбор данных видимым жизненным циклом.

Роль	Основные экраны	Назначение
Покупатель	ProductsFragment	Каталог, динамический поиск, категории, диапазон цены и Paging 3
Покупатель	MapFragment	Активные точки продаж, геолокация и список товаров точки
Покупатель	BasketFragment	Локальная корзина, количество, сумма и создание заказов
Покупатель	ProfileCustomerFragment	Профиль и переход к истории заказов
Фермер	MyProductFragment	Список товаров, DiffUtil, удаление жестом и отмена удаления
Фермер	AddNewProductFragment	Создание и редактирование товара, до трех изображений, multipart
Фермер	ProfileFarmerFragment	Профиль, выбор точки, активность, заказы и выход

6.2 Локальные данные

Хранилище	Данные	Требование
Room	Товары корзины и количество	Корзина сохраняется после закрытия приложения и доступна без сети
DataStore	Пользовательские настройки	Асинхронное типизированное хранение
Encrypted storage	JWT	Токен не должен храниться в открытом виде
Glide cache	Изображения товаров	Повторная загрузка должна использовать память и диск

7 Модель данных

Логическая модель ориентирована на PostgreSQL и разделяет пользователей, товары, изображения, точки продаж, заказы, позиции заказа, сообщения и отзывы. Связи должны обеспечиваться внешними ключами. Цена и название копируются в позицию заказа, чтобы история не менялась после редактирования товара.

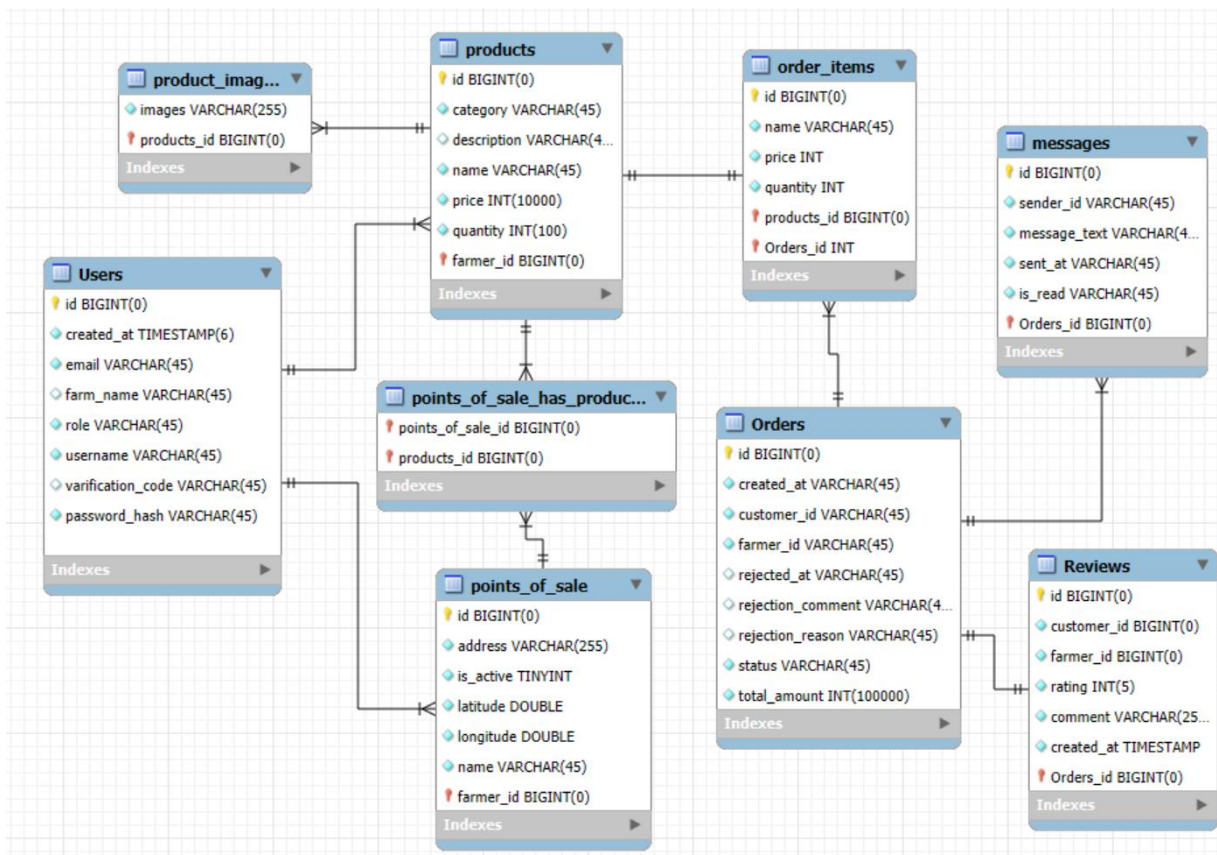


Рисунок 5 - Исходная ERD проекта

Ниже типы нормализованы для PostgreSQL. Перед миграцией их необходимо сверить с фактическими JPA-сущностями и миграциями. В исходной ERD часть временных, логических и ссылочных полей имеет тип VARCHAR или BIGINT, который не совпадает с назначением поля.

7.1 Сущность users

Атрибут	Тип	Обязателен	Назначение
id	BIGINT	Да	Первичный ключ
created_at	TIMESTAMP	Да	Дата создания
email	VARCHAR	Да	Уникальная почта
farm_name	VARCHAR	Нет	Название фермы
role	VARCHAR или ENUM	Да	CUSTOMER или FARMER

Атрибут	Тип	Обязателен	Назначение
username	VARCHAR	Да	Имя пользователя
verification_code	VARCHAR	Нет	Код подтверждения
password_hash	VARCHAR	Да	Хеш пароля

7.2 Сущность products

Атрибут	Тип	Обязателен	Назначение
id	BIGINT	Да	Первичный ключ
category	VARCHAR или ENUM	Да	Категория
description	VARCHAR или TEXT	Нет	Описание
name	VARCHAR	Да	Название
price	INTEGER	Да	Цена в минимальных денежных единицах
quantity	INTEGER	Да	Остаток
farmer_id	BIGINT	Да	Внешний ключ на users

7.3 Сущность points_of_sale

Атрибут	Тип	Обязателен	Назначение
id	BIGINT	Да	Первичный ключ
address	VARCHAR	Да	Адрес
is_active	BOOLEAN	Да	Активность точки
latitude	DOUBLE PRECISION	Да	Широта
longitude	DOUBLE PRECISION	Да	Долгота
name	VARCHAR	Да	Название
farmer_id	BIGINT	Да	Внешний ключ на users

7.4 Сущность product_images

Атрибут	Тип	Обязателен	Назначение
products_id	BIGINT	Да	Внешний ключ на products
images	VARCHAR	Да	Ссылка или путь к изображению

7.5 Сущность orders

Атрибут	Тип	Обязателен	Назначение
id	BIGINT	Да	Первичный ключ
created_at	TIMESTAMP	Да	Дата создания
customer_id	BIGINT	Да	Внешний ключ на users
farmer_id	BIGINT	Да	Внешний ключ на users
rejected_at	TIMESTAMP	Нет	Дата отклонения
rejection_comment	VARCHAR	Нет	Комментарий
rejection_reason	VARCHAR	Нет	Причина
status	VARCHAR или ENUM	Да	Состояние заказа
total_amount	INTEGER	Да	Сумма заказа

7.6 Сущность order_items

Атрибут	Тип	Обязателен	Назначение
id	BIGINT	Да	Первичный ключ
name	VARCHAR	Да	Снимок названия
price	INTEGER	Да	Снимок цены
quantity	INTEGER	Да	Количество
products_id	BIGINT	Да	Внешний ключ на products
orders_id	BIGINT	Да	Внешний ключ на orders

7.7 Сущность messages

Атрибут	Тип	Обязателен	Назначение
id	BIGINT	Да	Первичный ключ
sender_id	BIGINT	Да	Внешний ключ на users
message_text	VARCHAR или TEXT	Да	Текст сообщения
sent_at	TIMESTAMP	Да	Дата отправки
is_read	BOOLEAN	Да	Признак прочтения
orders_id	BIGINT	Да	Внешний ключ на orders

7.8 Сущность reviews

Атрибут	Тип	Обязателен	Назначение
id	BIGINT	Да	Первичный ключ
customer_id	BIGINT	Да	Автор
farmer_id	BIGINT	Да	Получатель отзыва
rating	INTEGER	Да	Оценка
comment	VARCHAR или TEXT	Да	Комментарий
created_at	TIMESTAMP	Да	Дата создания
orders_id	BIGINT	Да	Основание для отзыва

7.9 Ограничения целостности

- Email пользователя должен иметь уникальное ограничение.
- Товар, точка продаж и заказ должны ссылаться на пользователя требуемой роли.
- Количество и цена не могут быть отрицательными; рейтинг должен иметь ограниченный диапазон.
- У product_images требуется явный первичный ключ либо составное уникальное ограничение.
- Отзыв должен быть уникальным для пары покупатель и завершенный заказ.
- Удаление пользователя, товара и заказа должно иметь явную политику каскадирования или архивирования.

8 Программные интерфейсы

REST API использует JSON, а защищенные запросы передают JWT в заголовке Authorization по схеме Bearer. Добавление товара с изображениями использует multipart. Чат работает по WebSocket с протоколом STOMP; история загружается обычным GET-запросом.

Метод	Маршрут	Доступ	Назначение
POST	/api/auth/register	Открытый	Регистрация пользователя
POST	/api/auth/verif	Открытый	Подтверждение email по коду
POST	/api/auth/login	Открытый	Вход и получение JWT
GET	/api/customer/products	Покупатель	Каталог, поиск, фильтры и пагинация
GET	/api/customer/products/{id}/images	Покупатель	Изображения товара
POST	/api/customer/order	Покупатель	Создание бронирования
GET	/api/customer/profile	Покупатель	Профиль покупателя
GET	/api/customer/my-orders	Покупатель	История заказов
POST	/api/farmer/products/add	Фермер	Добавление товара
GET	/api/farmer/products/my	Фермер	Собственные товары
DELETE	/api/farmer/products/add/{id}/delete	Фермер	Удаление товара; путь требует сверки с кодом
GET	/api/farmer/profile	Фермер	Профиль фермера
GET	/api/farmer/my-orders	Фермер	Заказы фермера
PATCH	/api/farmer/orders/{id}/status	Фермер	Изменение статуса заказа
POST	/api/farmer/pos	Фермер	Создание точки продаж
GET	/api/farmer/pos/my	Фермер	Собственные точки продаж
PATCH	/api/farmer/pos/{id}/status	Фермер	Активация точки; в исходном отчете параметр обозначен как {is}

Метод	Маршрут	Доступ	Назначение
DELETE	/api/farmer/pos/{id}	Фермер	Удаление точки; в исходном отчете параметр обозначен как {is}
GET	/api/chat/{orderId}/history	Участник заказа	История сообщений
STOMP	/api/chat.send	Участник заказа	Отправка сообщения
STOMP	/api/chat.readAll	Участник заказа	Отметка сообщений прочитанными

8.1 Правила контракта

- Каждый защищенный REST-запрос должен проходить аутентификацию и ролевую авторизацию.
- DTO не должны раскрывать password_hash, verification_code и внутренние служебные поля.
- Ошибки валидации должны возвращаться единообразно с кодом, сообщением и перечнем полей.
- Параметры пагинации, сортировки и фильтрации должны быть зафиксированы в OpenAPI-описании.
- Изменение статуса заказа должно проверять допустимый переход и принадлежность заказа пользователю.
- WebSocket-соединение должно проверять JWT и право доступа к конкретному заказу.

9 Безопасность и конфигурация

Секреты нельзя хранить в документации, application.properties и Git. Параметры базы данных, почты, JWT и внешних SDK должны поступать из переменных окружения или защищенного хранилища секретов. Учетные данные, встречавшиеся в исходных материалах, необходимо заменить до публикации репозитория.

9.1 Безопасный пример application properties

```
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/farmer_market_db}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=validate
spring.mail.host=${MAIL_HOST:smtp.yandex.ru}
spring.mail.port=${MAIL_PORT:465}
spring.mail.username=${MAIL_USERNAME}
spring.mail.password=${MAIL_PASSWORD}
app.jwt.secret=${JWT_SECRET}
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=30MB
```

- Использовать BCrypt или другой адаптивный кодировщик паролей и не логировать пароль или токен.
- Ограничить CORS доверенными источниками и отключить подробные сообщения сервера в production.
- Задать срок жизни JWT и процедуру отзыва или обновления токена.
- Проверять MIME-тип, размер и содержимое загружаемых изображений.
- Добавить ограничение частоты для входа, верификации email и отправки сообщений.
- Хранить резервные копии PostgreSQL и регулярно проверять восстановление.

10 Сборка и запуск

10.1 Сервер

Компонент	Требование
Java	OpenJDK 17
СУБД	PostgreSQL 15
Сборка	Gradle Wrapper
Фреймворк	Версию Spring Boot необходимо брать из build.gradle и синхронизировать с документом
Локальный адрес	http://localhost:8080

1. Создать базу `farmer_market_db` и отдельного пользователя приложения с минимальными правами.
2. Задать `DB_URL`, `DB_USER`, `DB_PASSWORD`, `MAIL_USERNAME`, `MAIL_PASSWORD` и `JWT_SECRET` вне репозитория.
3. Выполнить `gradlew.bat clean build` в Windows или `./gradlew clean build` в Linux и macOS.
4. Запустить `java -jar build/libs/farmer-0.0.1-SNAPSHOT.jar`.
5. Проверить доступность `health endpoint`, подключение к PostgreSQL и отправку тестового письма.

10.2 Android клиент

1. Открыть проект в Android Studio с совместимой версией Android Gradle Plugin.
2. Добавить ключ Yandex MapKit и конфигурацию Firebase через локальные защищенные файлы.
3. Задать базовый URL сервера отдельно для `debug` и `release`.
4. Собрать `debug`-вариант через `gradlew.bat assembleDebug`.
5. Проверить регистрацию, карту, каталог, корзину, заказы и чат на поддерживаемой версии Android.

11 Тестирование

11.1 Функциональные сценарии покупателя

Сценарий	Ожидаемый результат
Открытие каталога	Загружаются и отображаются активные товары
Фильтр по цене и категории	Каталог показывает только соответствующие товары
Открытие карточки товара	Отображаются сведения и изображения
Добавление в корзину	Товар сохраняется в Room и счетчик обновляется
Создание заказа	Корзина передается серверу и делится по фермерам
Открытие карты	Отображаются активные точки продаж
Выбор точки	Показывается ассортимент выбранной точки
История заказов	Отображаются только заказы текущего покупателя

11.2 Функциональные сценарии фермера

Сценарий	Ожидаемый результат
Открытие списка товаров	Отображаются товары текущего фермера
Добавление или редактирование товара	Данные и изображения сохраняются, список обновляется
Удаление жестом	Показывается возможность отмены до фактического удаления
Подтверждение заказа	Статус меняется, покупатель получает уведомление
Добавление точки	Координаты и адрес сохраняются у текущего фермера
Активация точки	Выбранный ассортимент становится доступным покупателю

11.3 Модульные тесты

В материалах проекта описан класс `FarmerUnitTest`. Он проверяет преобразование названий категорий в `ProductCategory` с возвратом `OTHER` для неизвестного значения и проверку сложности пароля: не менее шести символов, наличие цифры, строчной и прописной буквы. Эти тесты следует дополнять проверками переходов статуса заказа, разбиения корзины, ролевого доступа и расчетов итоговой суммы.

11.4 Результаты нагрузочного теста

Сценарий	Запросы	Среднее мс	Максимум мс	Ошибки проц	Запросов в сек
Профиль фермера	1500	4006	8881	0,00	91,3
История заказов	1500	4003	9703	0,00	102,9
Каталог без фильтра	1500	2006	7823	0,00	90,8
Каталог с фильтром	1500	1225	6946	0,00	91,1
Итого	6000	2810	9703	0,00	357,0

Тест на 1500 активных пользователей завершился без ошибок и достиг суммарной интенсивности 357 запросов в секунду. При этом средний отклик всех измеренных эндпоинтов превысил целевое значение менее одной секунды. До приемки по производительности необходимо зафиксировать профиль нагрузки, измерять p95 и p99, оптимизировать медленные запросы и повторить испытание на production-подобной среде.

12 Сопровождение и публикация

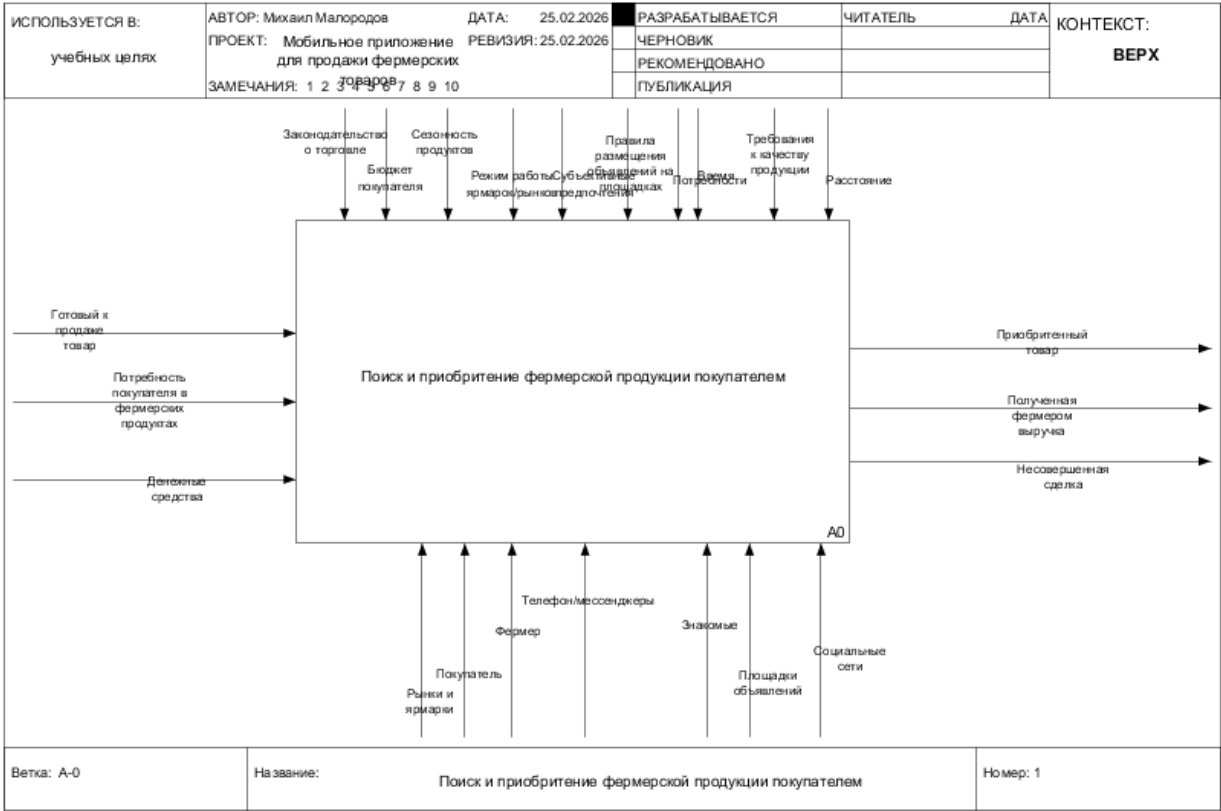
12.1 Правила обновления документации

- Изменение маршрута API сопровождается обновлением таблицы интерфейсов и теста контракта.
- Изменение схемы данных сопровождается миграцией, обновлением ERD и описания сущности.
- Изменение бизнес-процесса сопровождается новым экспортом PNG из редактируемого источника.
- Секреты и реальные учетные данные не допускаются ни в документе, ни в примерах конфигурации.
- Версия документа и дата изменения фиксируются в коммите вместе с изменением системы.

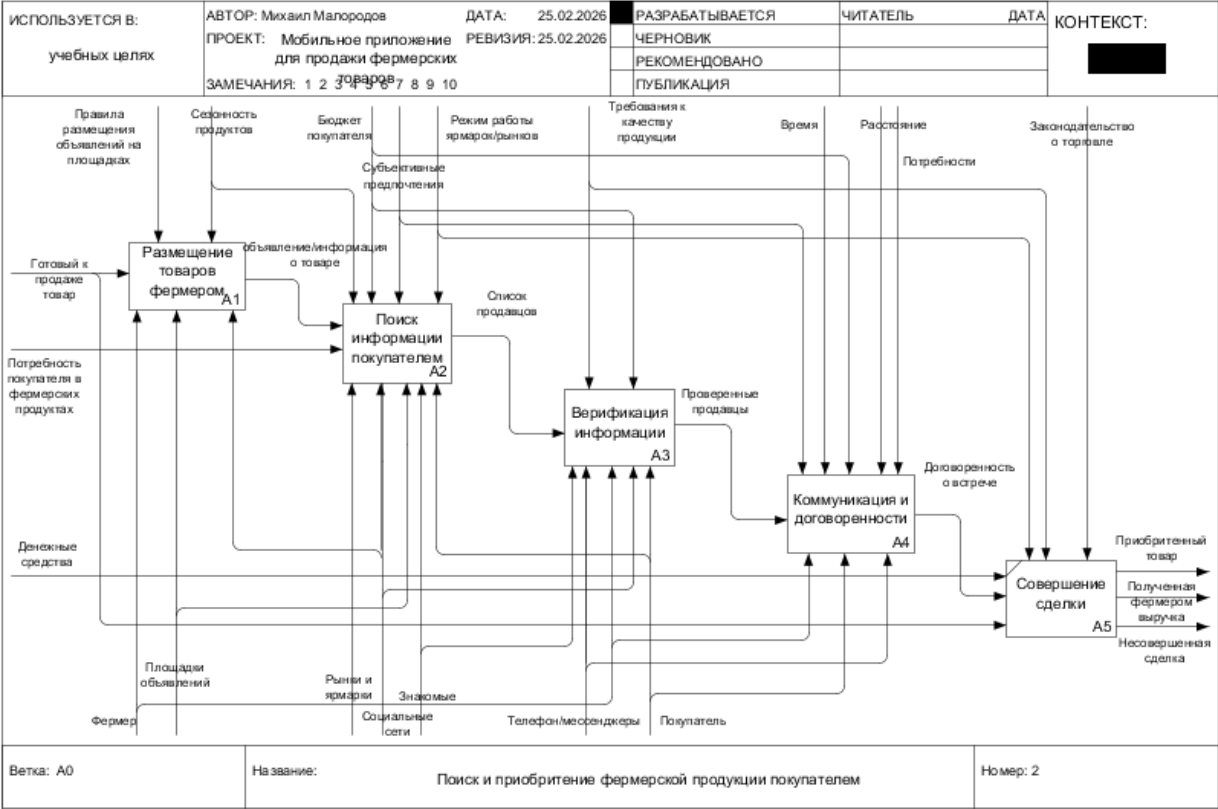
12.2 Пункты для сверки с исходным кодом

Область	Что требуется проверить	Причина
Секреты	Сменить опубликованные учетные данные и очистить историю Git	В исходных отчетах присутствуют реальные значения
Android	Сопоставить заявленную версию Android с minSdk	Версия ОС и API level в материалах указаны несогласованно
Spring Boot	Взять точную версию из build.gradle	В отчете указано значение, требующее проверки
API точек	Проверить параметр {id} и фактические пути	В отчете встречается обозначение {is}
API товаров	Зафиксировать маршруты обновления товара	Функция описана, но отдельный маршрут не приведен
ERD	Синхронизировать типы ссылок, времени и логических полей	Изображение ERD и таблицы спецификации расходятся
product_images	Определить первичный ключ или уникальное ограничение	В исходной спецификации ключ отсутствует
Эксплуатация	Добавить health endpoint, миграции, резервное копирование и мониторинг	Нужно для воспроизводимого развертывания

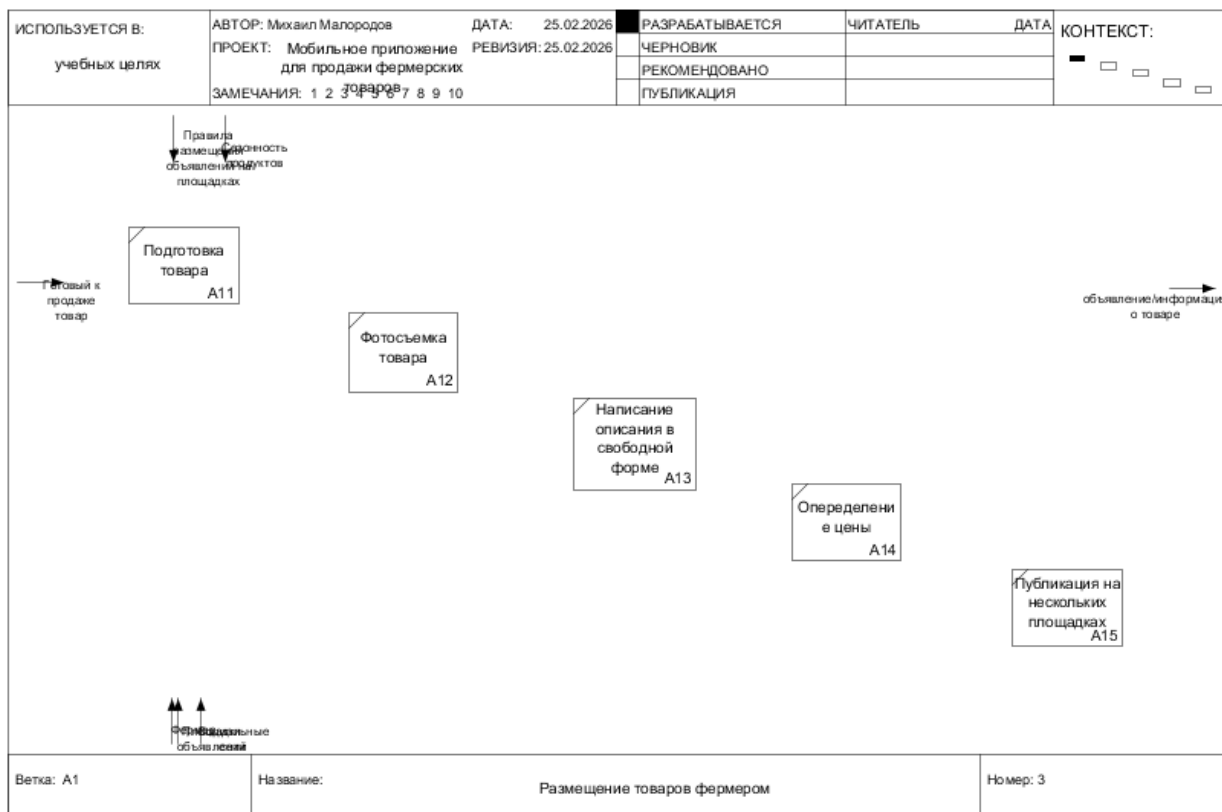
13 Приложение диаграммы



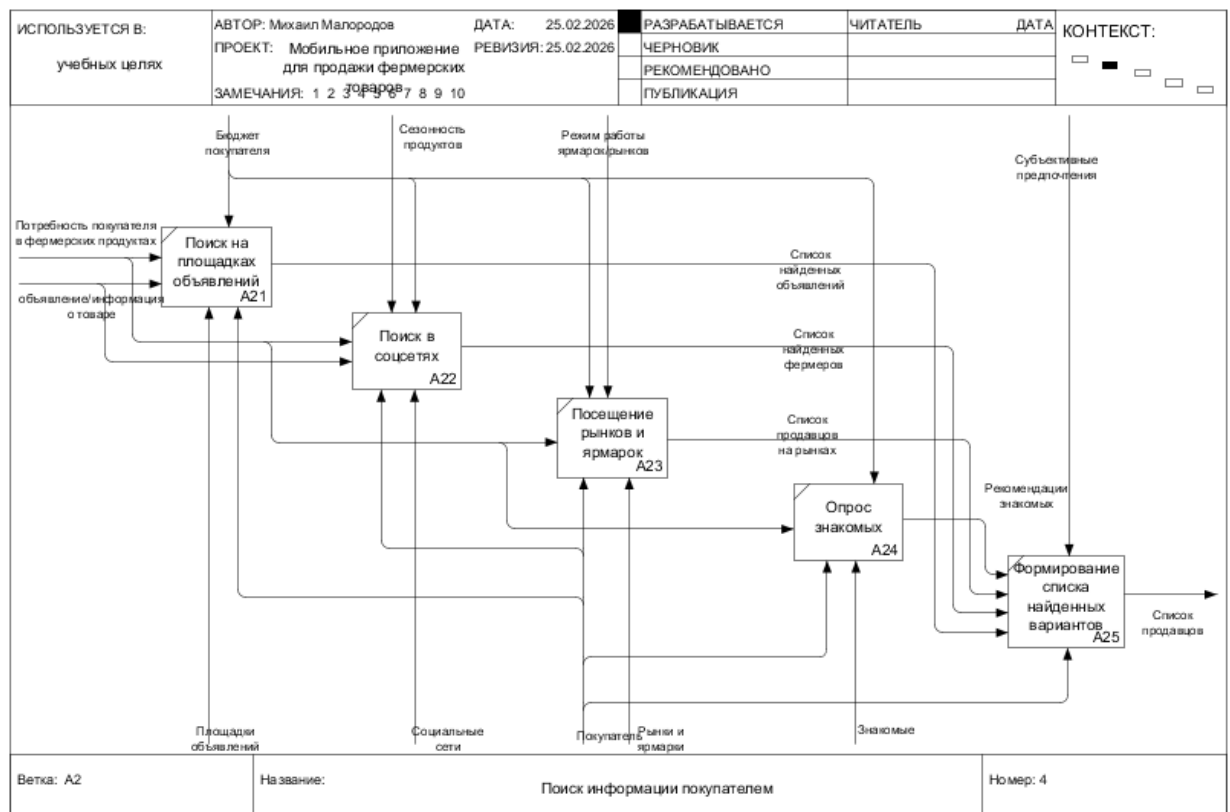
Приложение А - Контекст процесса AS IS



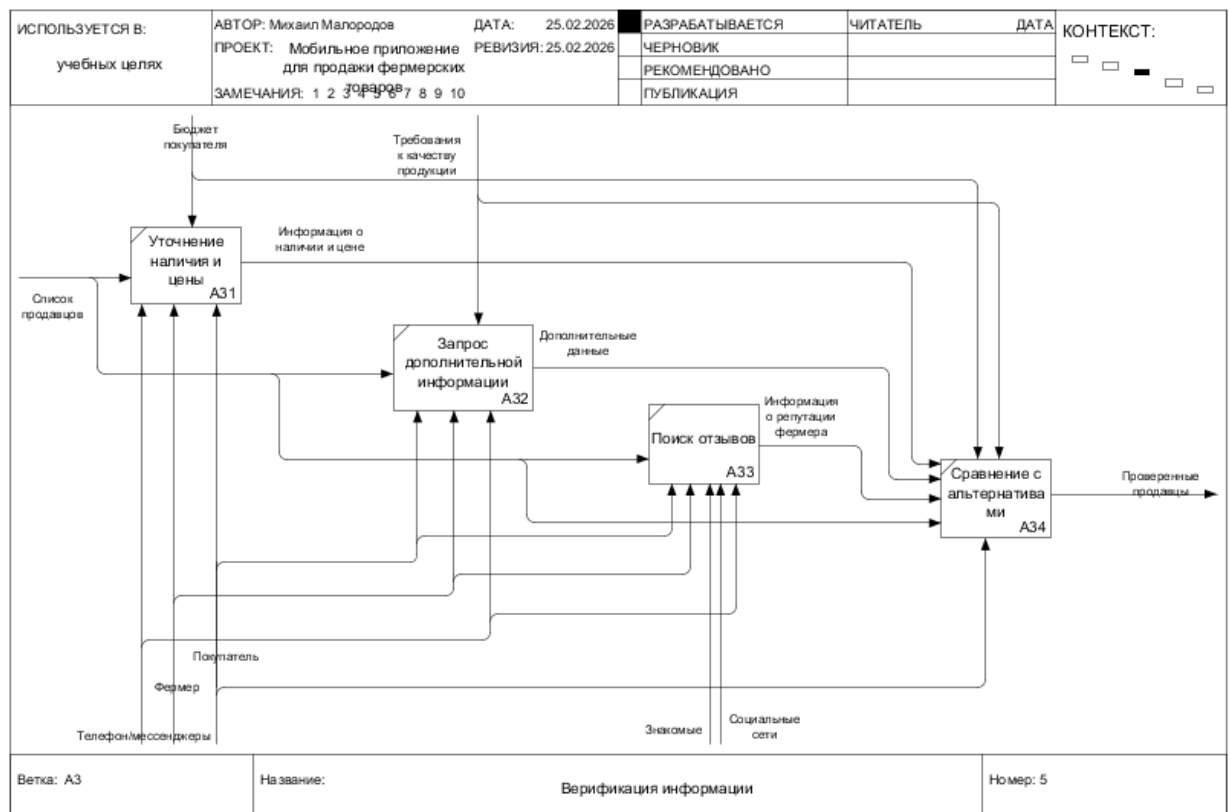
Приложение Б - Декомпозиция процесса AS IS



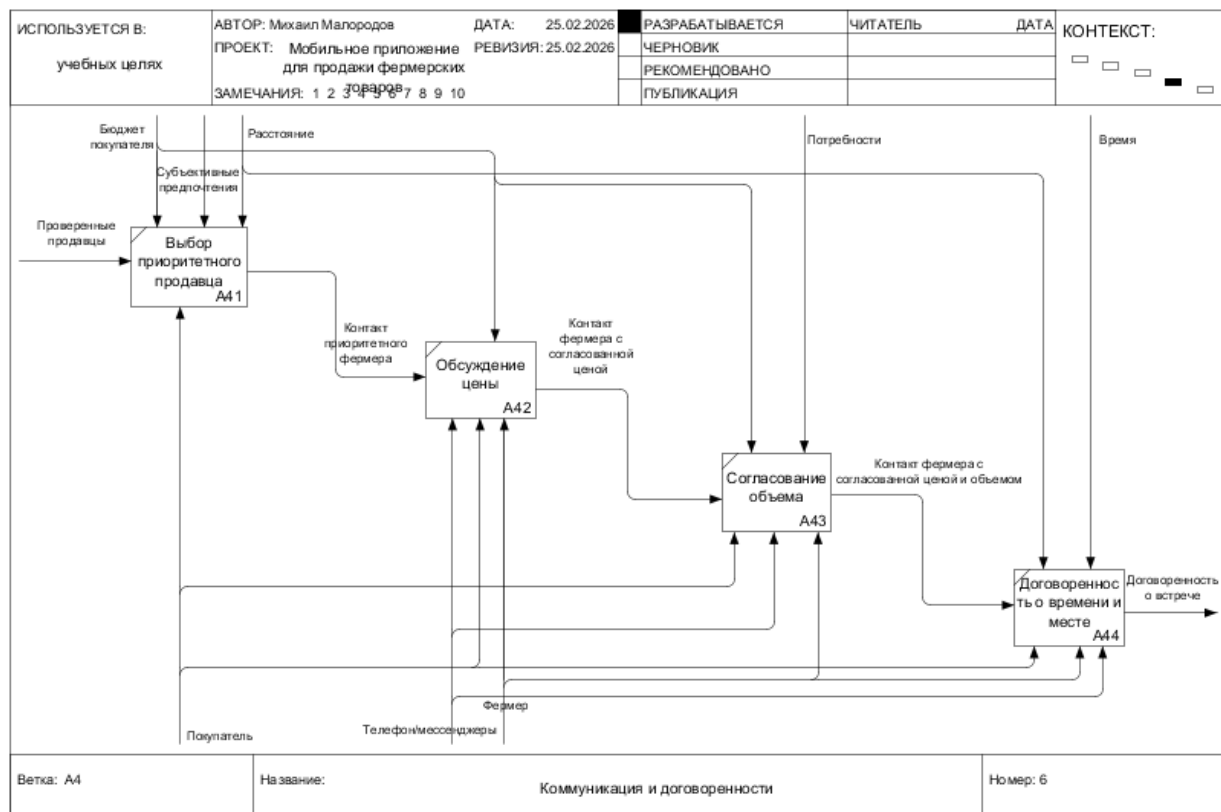
Приложение В - Размещение товара фермером



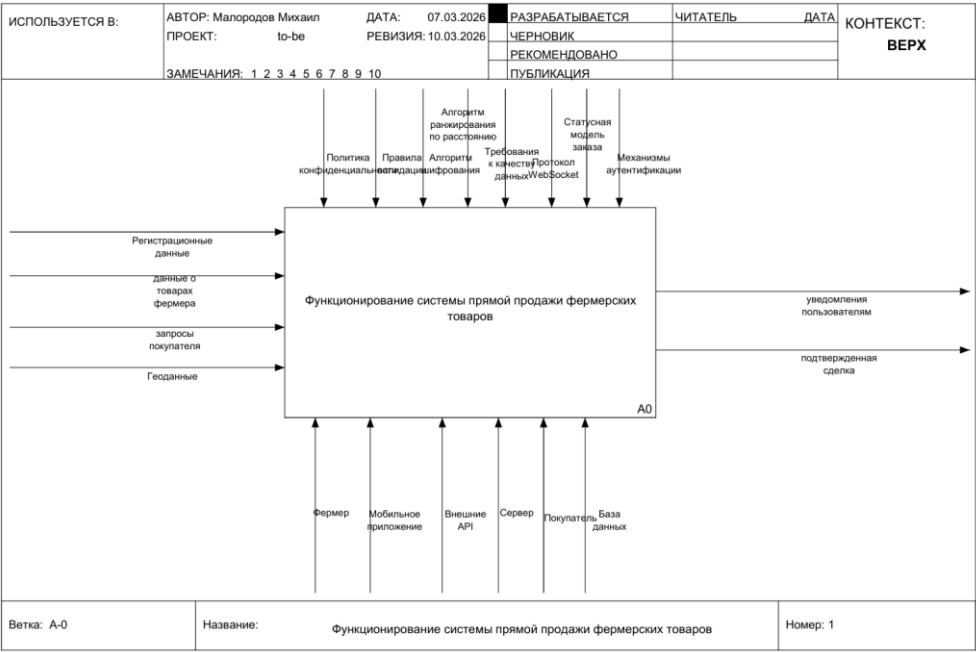
Приложение Г - Поиск информации покупателем



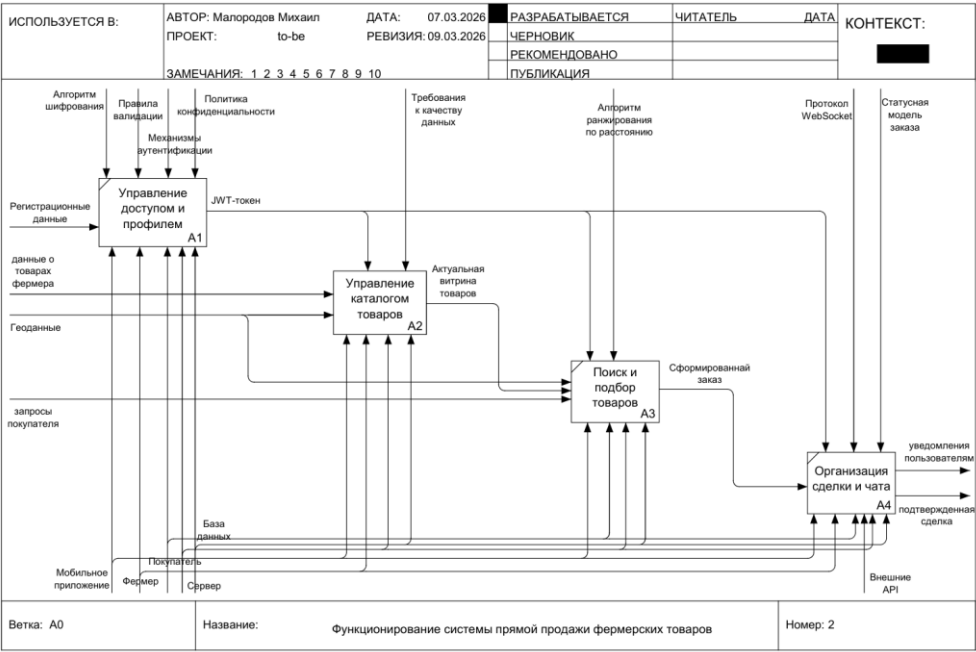
Приложение Д - Проверка информации



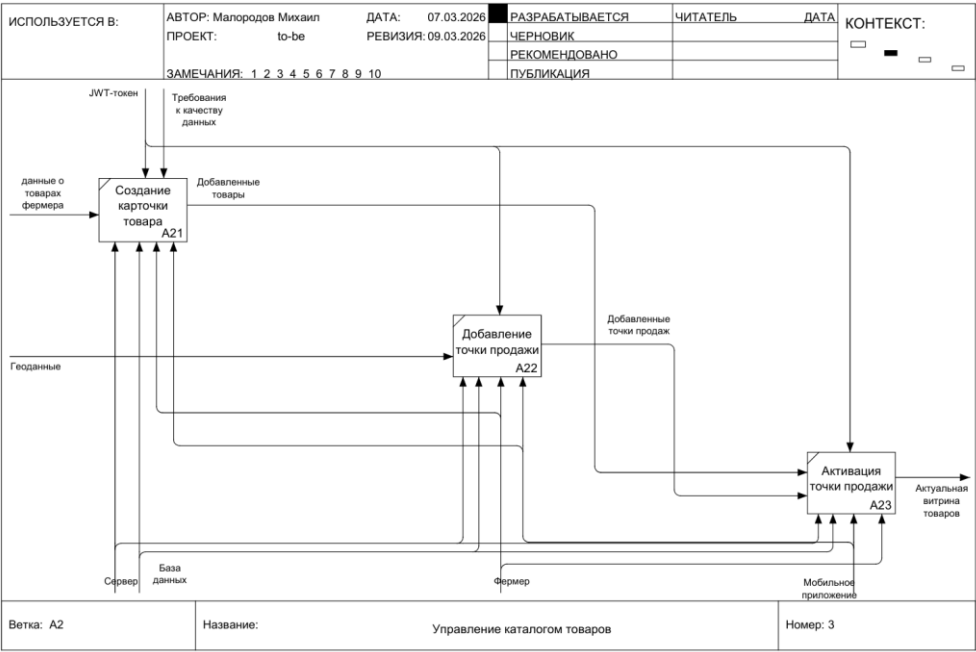
Приложение Е - Коммуникация и договоренности



Приложение Ж - Контекст процесса TO BE



Приложение И - Декомпозиция процесса TO BE



Приложение К - Управление каталогом TO BE